UNIVERSIDAD NACIONAL DE LA PLATA

FACULTAD DE INFORMÁTICA

TALLER DE PROYECTO 2

INGENIERÍA EN COMPUTACIÓN

DETECCIÓN DE PRESENCIA [M1]

INFORME DE AVANCE - 11/10/2026

Grupo de Trabajo

Bernardo Etcheto - 03775/3

Emiliano Paoloni - 03658/8

Francisco Marco Del Pont - 02323/1

Docente responsable: Merlina

Índice

1. Introducción 1
2. Objetivo 1
3. Esquema Gráfico del Proyecto 3
4. Identificación de Partes 6
4.1. Alimentación del sistema 7
4.2. Software y procesamiento de datos 7
4.3. Sistema de visualización web 8
5. Avances Cronológicos de Tareas 9
5.1. Informe de Avance/Entregable de Octubre 10
5.2. Avance/Entregable de Noviembre 10
6. Grado de Avance, Problemas y Soluciones 10
6.1. Firmware desarrollado para prueba del sensor PIR 10
6.2. Firmware desarrollado para prueba de obtención de señal CSI 12
6.3. Interfaz Web 13
7. Documentación Relacionada 15
8. Documentación en Video 16
9. Bibliografía 16

1. Introducción

En el ámbito de los sistemas de automatización, la seguridad y la Internet de las Cosas (IoT), la detección de presencia en espacios interiores es una función crítica. Tradicionalmente, esta tarea se resuelve utilizando sensores infrarrojos pasivos (PIR). Sin embargo, esta tecnología presenta limitaciones operativas significativas: requiere línea de visión directa, posee zonas ciegas y frecuentemente falla al detectar personas estáticas, lo que genera falsos negativos.

Una alternativa prometedora es el uso de la Información de Estado del Canal (CSI, Channel State Information), una técnica que analiza las características del canal inalámbrico —como la amplitud, la fase y el retardo de la señal— para inferir cambios en el entorno físico [1]. El principio subyacente es el efecto multipath de la propagación de señales de radio: cuando un objeto o persona se mueve en el espacio, las rutas de reflexión de la señal cambian, alterando la amplitud y fase de las señales superpuestas y provocando fluctuaciones en los datos de CSI [1]. Esta sensibilidad permite detectar no solo movimientos grandes (caminar, correr), sino también acciones sutiles en entornos estáticos, como la respiración.

Esta investigación busca evaluar empíricamente la viabilidad y las ventajas de la detección por CSI frente a los métodos convencionales. Se analizarán distintos escenarios de transmisión (uso de routers comerciales vs. redes de nodos dedicados) y se desarrollará una infraestructura completa de telemetría mediante el protocolo MQTT para la evaluación de métricas en tiempo real. Para ello, se utilizará la plataforma ESP32 de Espressif, que ofrece soporte nativo para CSI en toda su serie de microcontroladores y un ecosistema robusto que integra esta tecnología con dispositivos IoT existentes [2].

2. Objetivo

El objetivo principal del proyecto es evaluar empíricamente y comparar la eficacia de la detección de presencia humana en interiores utilizando el análisis de la Información de Estado del Canal (CSI) de señales Wi-Fi frente a sensores de movimiento infrarrojos convencionales (PIR). Adicionalmente, se busca desarrollar e implementar una infraestructura de telemetría completa que permita la visualización centralizada de estas métricas de detección mediante el protocolo MQTT y un dashboard web.

Requerimientos Principales:

- **Captura de datos de línea base:** Lectura y registro de estados de movimiento utilizando un módulo ESP32 y un sensor PIR (Caso 1).
- **Extracción de CSI:** Captura de datos CSI en crudo desde la capa física del ESP32 conectado como cliente a un router comercial (Caso 2).
- **Procesamiento de señal y Criterios de Éxito:** Filtrado de ruido, reducción de dimensionalidad y aplicación de algoritmos para determinar la presencia/ausencia. Se definen los siguientes criterios de validación para comparar contra la línea base (Caso 1):
    - **Reducción de falsos negativos:** Disminuir en al menos un **80%** los falsos negativos del sensor PIR en escenarios de presencia estática, como por

ejemplo una persona sentada sin movimientos bruscos durante más de 60 segundos.
    - **Detección sin línea de visión:** Lograr una precisión de detección superior al **85%** cuando la persona se encuentra detrás de obstáculos no estructurales (pared de yeso, puertas de madera), que es un escenario donde el PIR tiene 0% de efectividad.
    - **Latencia de procesamiento:** El algoritmo debe procesar la ventana de datos CSI y determinar el estado de presencia en un tiempo inferior a **500 ms**.

- **Telemetría IoT:** Envío del estado de detección ("presencia detectada" / "espacio vacío") y métricas de desempeño desde los ESP32 hacia un broker MQTT.

• **Visualización Web:** Integración de un dashboard que reciba del backend los estados de detección y las métricas mediante WebSocket y REST, para comparar los tres casos.

Se establecen los siguientes parámetros visuales y operativos tentativos:

- **Frecuencia de actualización:** El dashboard debe reflejar los cambios de estado con una latencia máxima de 5 a 10 segundos.
- **Métricas a comparar en tiempo real:**
    - **Estado Binario de Presencia:** Indicadores en paralelo del estado actual del PIR (Movimiento/Vacío) vs. CSI (Presencia/Vacío).
    - **Línea de tendencia de detección:** Un gráfico temporal que muestre las "caídas" de detección del PIR (cuando la persona se queda quieta) contrastado con el mantenimiento continuo de la detección del CSI.
    - **Varianza del Canal:** Un gráfico lineal mostrando la perturbación de la amplitud de las subportadoras CSI en crudo, permitiendo visualizar el "ruido" que genera una persona al moverse o respirar.

Requerimientos Secundarios:

- **Red dedicada (Caso 3):** Implementación de una topología de red aislada compuesta por dos nodos ESP32 independientes (uno como emisor/Access Point y otro como receptor/Station). Se plantea como funcionalidad secundaria extraer paquetes CSI en este entorno controlado para analizar si la eliminación del router comercial (y su tráfico asociado) mejora la resolución de la detección respecto a los casos anteriores.

3. Esquema Gráfico del Proyecto

Se muestran los esquemas gráficos de las 3 configuraciones involucradas en el proyecto:

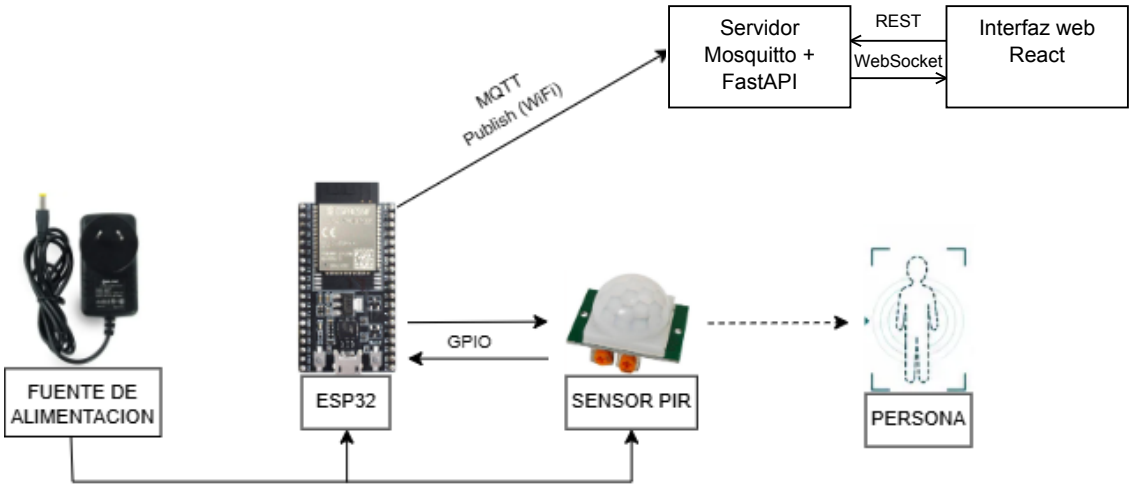


Figura 1: Configuración del caso base

En la Figura 1 se observa el caso base: un ESP32, un sensor PIR y un servidor. El PIR detecta cambios en la radiación infrarroja del ambiente. El ESP32 lee su salida digital y publica la lectura por Wi-Fi mediante MQTT hacia Mosquitto. En el servidor, el backend Python/FastAPI se suscribe al tópico, valida los mensajes y almacena las lecturas en SQLite. Finalmente, la interfaz React recibe los estados en vivo mediante WebSocket y consulta el historial mediante REST. El navegador se comunica con el backend; no se suscribe directamente al broker MQTT.

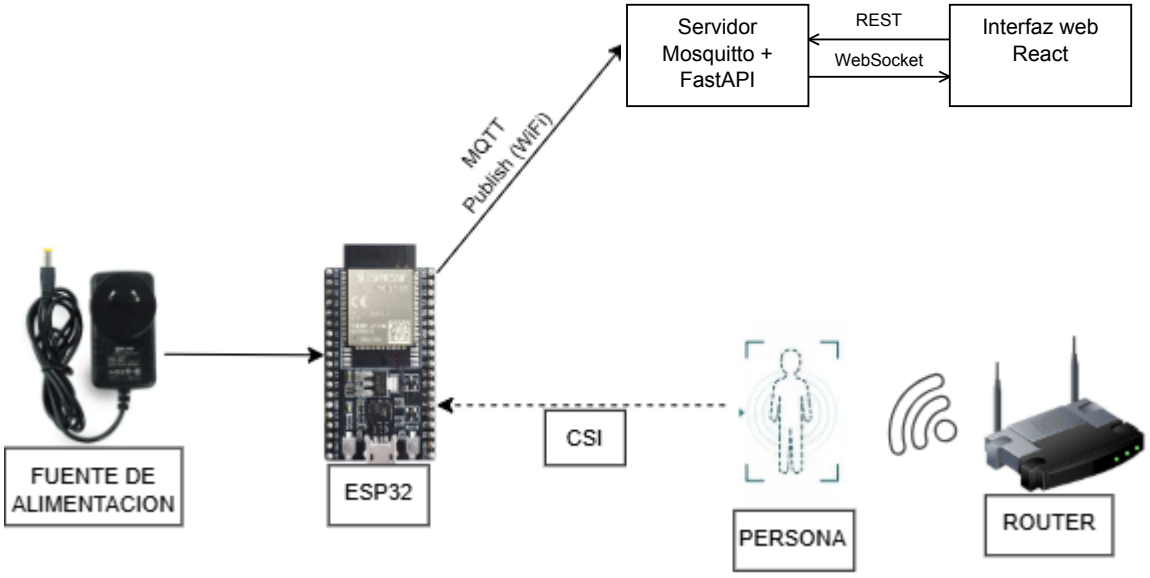


Figura 2: Configuración del caso 2

La Figura 2 representa la configuración con infraestructura existente: un ESP32 receptor, un router Wi-Fi y un servidor. El ESP32 obtiene información CSI de los paquetes recibidos desde el router. La arquitectura prevista envía estos datos mediante MQTT hacia Mosquitto. El backend recibe las muestras, realiza el procesamiento de la señal y obtiene el estado de detección. Luego guarda los datos y comunica los resultados al dashboard mediante WebSocket y REST. La primera prueba de captura CSI descrita en la sección 6.2 se realizó por Serial; su integración completa con MQTT y la detección todavía requiere validación experimental.

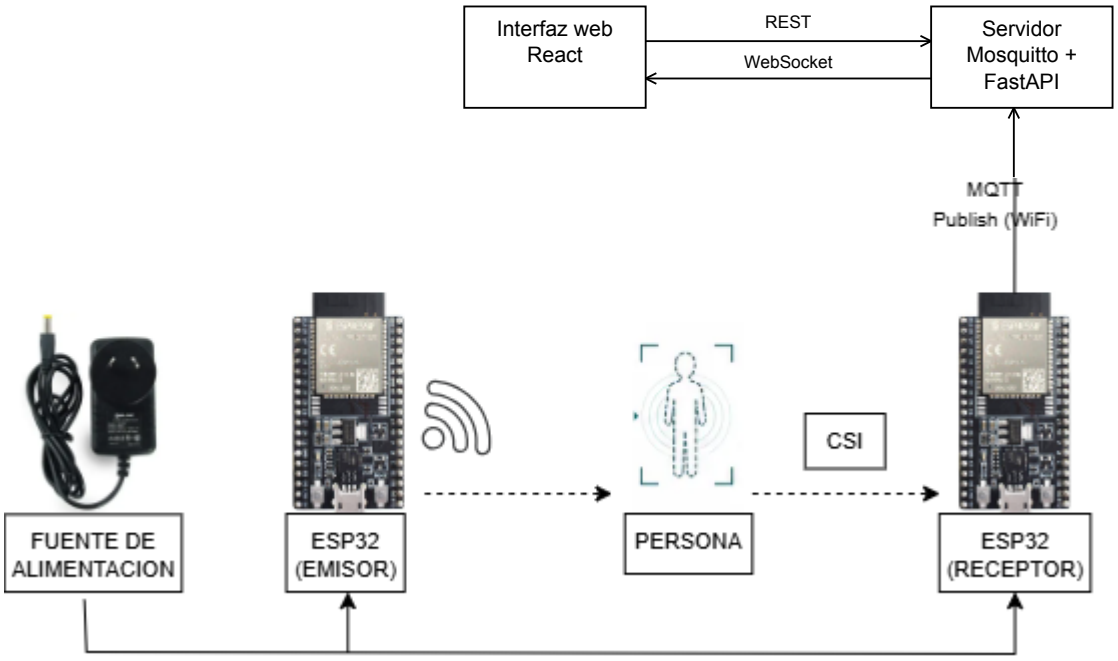


Figura 3: Configuración del caso 3

El Figura 3 representa la topología del caso 3. Las partes físicas son: dos microcontroladores (ESP32_1 como Emisor y ESP32_2 como Receptor) y un Servidor. El ESP32_1 se configura como Access Point para inyectar paquetes controlados al medio. El ESP32_2, operando en modo Station, recibe estos paquetes, extrae los datos CSI y los transmite en crudo al Servidor. Luego el flujo de datos es igual al caso 2.

4. Identificación de Partes

Para la implementación y evaluación de los tres casos propuestos se utilizarán los siguientes componentes:

Elemento	Modelo de referencia	Descripción	Cant.	Precio de referencia	Tensión de alimentación	Corriente / consumo	Imagen
Microcontrolador ESP32	NodeMCU ESP32S / ESP32-WROOM-32, 38 pines, Micro-USB	Utilizado para la adquisición de datos y la comunicación Wi-Fi. En el Caso 1 se empleará un ESP32 conectado al sensor PIR; en el Caso 2 un ESP32 funcionará como receptor CSI conectado a una red Wi-Fi existente; y en el Caso 3 se utilizarán dos ESP32 configurados como Access Point y Station.	5	$20.600 c/u [3]	5 V mediante USB. El microcontrolador trabaja internamente a 3,3 V.	Para el diseño conviene disponer de al menos 500 mA por placa. El consumo real depende de la actividad Wi-Fi y puede presentar picos superiores al consumo medio.	
Sensor PIR	HC-SR501	Utilizado como sistema de referencia para la detección convencional de movimiento. Detecta variaciones de radiación infrarroja producidas por el movimiento de personas dentro de su campo de detección.	1	$3.800 [4]	5 V CC para la implementación prevista.	Consumo muy bajo; típicamente del orden de decenas de µA en reposo.	
Router Wi-Fi	TP-Link TL-WR840N (referencia)	Utilizado en el Caso 2 como infraestructura de comunicación Wi-Fi, permitiendo generar y transportar el tráfico sobre el cual el ESP32 realizará la captura de información CSI.	1	$41.000 [5]	9 V CC mediante adaptador externo conectado a 220 V CA	Adaptador de hasta aprox. 0,6 A (máximo teórico ≈ 5,4 W).	
Computadora	PC / Notebook disponible – modelo a definir	Utilizada para programar los ESP32, recibir los datos obtenidos, procesar las señales CSI, ejecutar los algoritmos de detección, alojar el broker MQTT y ejecutar el sistema de visualización web.	1	Sin costo adicional si se utiliza un equipo disponible	Alimentación propia del equipo.	Depende del modelo de computadora utilizado.	
Cable USB	USB-A a Micro-USB, 1 m	Utilizado para alimentar, programar y establecer la comunicación serie entre los módulos ESP32 y la computadora.	2	$5500 [6]	Transporta 5 V desde el puerto USB.	No posee consumo propio significativo; conduce la corriente requerida por el ESP32.	
Cables Dupont	Pack de cables Dupont, 20 cm, macho-hembra	Utilizados para realizar las conexiones eléctricas entre el sensor PIR, la protoboard y los pines correspondientes del ESP32.	1 juego	$5.000 [7]	No poseen tensión de alimentación propia.	No presentan consumo propio; únicamente conducen señales y alimentación.	
Fuente de alimentación	Fuente switching regulada 5 V CC / 2 A	Utilizada opcionalmente para alimentar de forma independiente el sensor PIR durante las pruebas, en caso de que la alimentación directa desde el ESP32 introduzca ruido en la señal. Permite evaluar si una alimentación separada mejora la calidad de la medición.	1 (opcional)	$10.000 [8]	Entrada: 220 V CA. Salida: 5 V CC.	Capacidad máxima aproximada: 2 A (10 W).	

4.1. Alimentación del sistema

Durante el desarrollo y la implementación del sistema, los módulos ESP32 serán alimentados mediante conexión USB desde la computadora. Esta conexión suministra 5 V a la placa, que dispone de regulación interna para obtener los 3,3 V necesarios para el funcionamiento del microcontrolador.

En la configuración correspondiente al Caso 1, el ESP32 será alimentado mediante USB y el sensor PIR obtendrá inicialmente su alimentación desde uno de los pines disponibles en la placa, respetando la tensión de funcionamiento especificada para el modelo utilizado. Debido a que esta forma de alimentación puede introducir ruido adicional en la señal, durante las pruebas se contempla, en caso de ser necesario, la utilización de una fuente de alimentación independiente para el sensor. No obstante, debido a la robustez del PIR, esta alternativa se considera opcional.

En el Caso 2, el ESP32 encargado de la captura CSI será alimentado mediante conexión USB desde la computadora, mientras que el router Wi-Fi se conectará directamente a la red eléctrica mediante su fuente de alimentación correspondiente.

En el Caso 3, los dos módulos ESP32 utilizados respectivamente como Access Point y Station serán alimentados individualmente mediante sus respectivas conexiones USB.

Debido a las características del prototipo, no se prevé inicialmente el desarrollo de un circuito de alimentación específico. Se utilizarán las fuentes disponibles en los dispositivos y equipos empleados durante los ensayos, contemplándose una alimentación independiente para el sensor PIR únicamente si las pruebas experimentales muestran que resulta necesaria para reducir el ruido de la señal.

4.2. Software y procesamiento de datos

El sistema estará compuesto por diferentes procesos de software encargados de la adquisición, procesamiento, transmisión y posterior visualización de los datos correspondientes a los tres casos de estudio.

En una primera etapa se realizará la adquisición de los datos. Para el caso de referencia, el ESP32 obtendrá el estado proporcionado por el sensor PIR. En los casos basados en CSI, los ESP32 capturarán la información de estado del canal correspondiente a las señales Wi-Fi recibidas.

Los datos CSI obtenidos serán posteriormente enviados a la computadora para su procesamiento. Esta etapa incluirá el preprocesamiento de los datos, el filtrado de ruido y el análisis de las variaciones producidas en la señal, con el objetivo de determinar la presencia o ausencia de personas en el ambiente.

A partir de la información procesada se generará un estado de detección para cada método. Estos resultados permitirán posteriormente comparar el comportamiento de las distintas configuraciones bajo condiciones experimentales equivalentes.

Además, se calcularán métricas destinadas a evaluar cuantitativamente el desempeño de cada método, considerando indicadores como la tasa de detección, falsos positivos, falsos negativos y latencia de detección.

Para la comunicación entre los nodos ESP32 y el backend se utilizará MQTT con un broker central. Esta organización permite

desacoplar la adquisición del procesamiento y la visualización. El backend transmite los estados al navegador mediante WebSocket y ofrece el historial mediante REST.

El flujo general comienza con la adquisición de datos PIR o CSI en el ESP32 y su envío por MQTT. En la computadora, el backend valida las muestras, realiza el procesamiento correspondiente y guarda los registros. Finalmente, los estados y las métricas disponibles se presentan en la interfaz web mediante REST y WebSocket.

4.3. Sistema de visualización web

El sistema web tendrá como función centralizar la visualización de los resultados obtenidos durante los ensayos y facilitar la comparación entre las distintas configuraciones implementadas. La interfaz recibirá los datos generados por el sistema y permitirá observar tanto el estado de detección como la evolución temporal de los datos y las métricas calculadas.

El dashboard deberá permitir comparar directamente los tres casos de estudio: el método de referencia basado en el sensor PIR, la detección mediante CSI utilizando un router Wi-Fi y la detección mediante CSI utilizando una red dedicada formada por dos nodos ESP32.

Entre las principales funcionalidades previstas se encuentran la visualización del estado de presencia o ausencia, la representación gráfica de los datos obtenidos durante los ensayos, el seguimiento de su evolución temporal, la presentación de métricas de desempeño y la comparación directa entre los tres métodos de detección. Además, se contempla incorporar una visualización del procesamiento de la señal CSI, mostrando la señal recibida, los filtros o transformaciones aplicados y la señal resultante utilizada para la detección.

Para la implementación del sistema de visualización se analizaron cuatro alternativas:

• ThingsBoard: plataforma orientada a sistemas IoT que permite integrar dispositivos y datos de telemetría mediante MQTT. Ofrece herramientas para construir dashboards y visualizar estados y métricas en tiempo real.

• Grafana: plataforma especializada en la visualización de métricas y series temporales. Permite construir paneles comparativos y gráficos históricos, aunque requiere una capa adicional de almacenamiento, como una base de datos de series temporales.

• Node-RED: entorno de programación visual basado en flujos que permite integrar dispositivos, brokers MQTT y sistemas de visualización de forma rápida. Es una alternativa adecuada para el prototipado y la validación inicial del sistema.

• Interfaz web propia: desarrollo de una aplicación diseñada específicamente para las necesidades del proyecto. Esta alternativa permitiría mayor flexibilidad para organizar las visualizaciones, representar el procesamiento de la señal y comparar los resultados de los distintos casos de estudio.

Tras analizar estas alternativas, se eligió una interfaz web propia desarrollada con React y TypeScript, utilizando Vite como herramienta de desarrollo. El backend se implementó en Python con FastAPI. Esta combinación permite adaptar las vistas a los tres casos y mantener el procesamiento de señales, la validación y el almacenamiento separados de la presentación.

La comunicación entre los dispositivos ESP32 y el sistema de procesamiento se realizaría mediante MQTT, utilizando un broker como Mosquitto. El backend recibiría los datos publicados por los ESP32, realizaría el procesamiento correspondiente y enviaría los

resultados al frontend. Para la actualización de datos en tiempo real entre el backend y la interfaz se contempla el uso de WebSocket.

Para representar señales y métricas se utiliza Apache ECharts. La persistencia inicial se implementó con SQLite, que permite almacenar lecturas y ensayos sin instalar un servidor de base de datos adicional. InfluxDB queda como una alternativa futura, únicamente si el volumen de series temporales lo justifica.

La arquitectura implementada para la interfaz web propia es:

ESP32 → MQTT → Broker Mosquitto → Backend Python/FastAPI → WebSocket → Frontend React

La interfaz web propia permite adaptar la visualización a los ensayos y comparar los métodos de detección. En esta etapa se verificó el recorrido completo del PIR, desde el sensor hasta la web mediante MQTT. La integración de las mediciones CSI y la comparación cuantitativa de los tres casos se completarán en las siguientes etapas.

5. Avances Cronológicos de Tareas

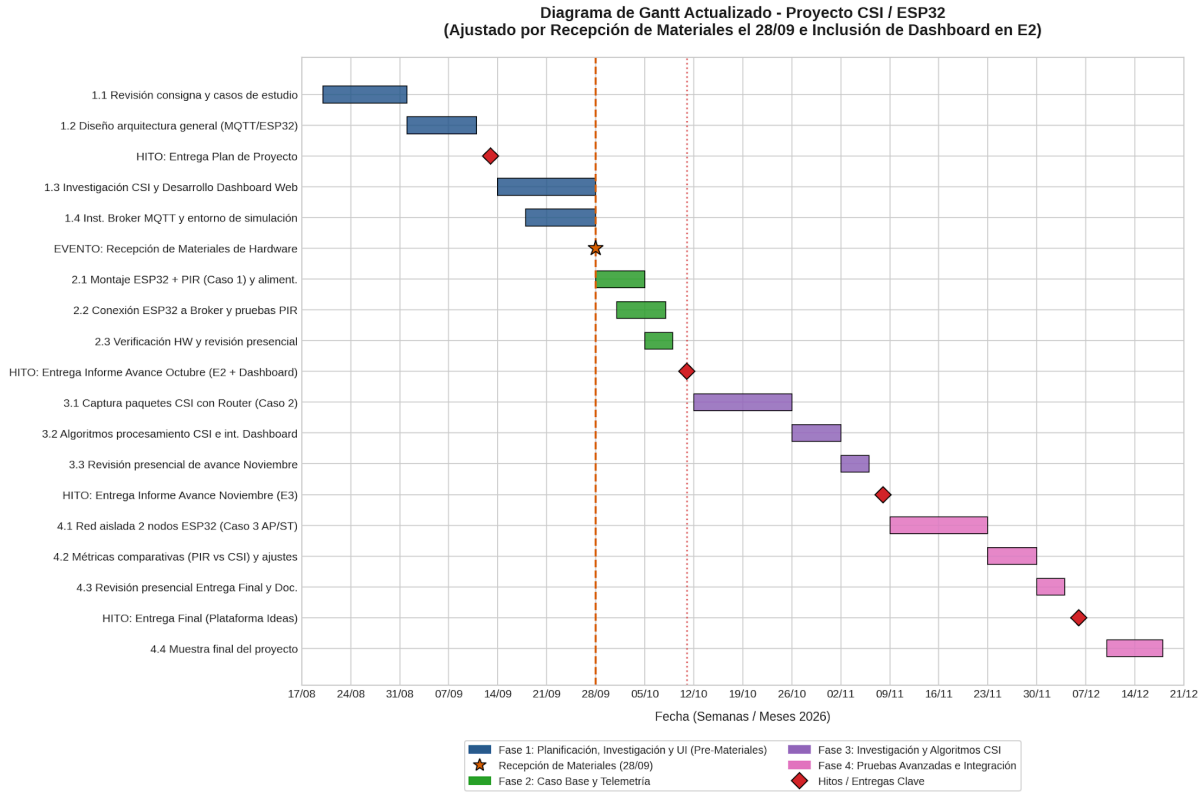


Figura 4: Cronograma

5.1 Informe de Avance/Entregable de Octubre

Para la fecha de entrega del Informe de Avance de Octubre (11/10), planificamos tener implementado y funcional el **Caso 1 (Caso Base)**. Esto incluye el montaje físico del módulo ESP32 junto con el sensor PIR, la correcta lectura de los estados de movimiento y la implementación inicial de la infraestructura de telemetría, logrando que el ESP32 publique exitosamente los estados de detección hacia el broker MQTT. Todo el esquema de hardware y los primeros scripts de conexión estarán documentados en la bitácora

5.2 Avance/Entregable de Noviembre

Para la entrega del Informe de Avance de Noviembre (08/11), el objetivo es integrar el Caso 2 y completar su validación en el dashboard. Se espera capturar CSI desde el router comercial, transmitir las muestras y visualizarlas junto con el PIR. Mosquitto recibirá la telemetría por MQTT y el backend enviará los resultados a la interfaz mediante WebSocket y REST. Luego se realizarán el ajuste de filtros, los ensayos comparativos y la exploración del Caso 3 (red dedicada).

6. Grado de Avance, Problemas y Soluciones

Hasta la fecha se realizó el montaje del Caso 1 y se comprobaron los dos estados de salida del PIR (0 y 1), tanto por Serial como en la interfaz web mediante MQTT. Además, se llevó a cabo una primera prueba de recepción de CSI con un ESP32 y un router Wi-Fi, correspondiente al Caso 2. Estos avances validan la adquisición y comunicación iniciales; aún faltan ensayos controlados para medir el desempeño de detección.

6.1 Firmware desarrollado para prueba del sensor PIR

El firmware se desarrolló en C++ para ESP32 con el framework Arduino. Se utilizó el sensor PIR HW-416-B: VCC a VIN/5 V, GND a GND y OUT a GPIO27. La placa se alimenta por USB. El modelo HC-SR501 de la tabla de partes es una referencia; el módulo utilizado en esta prueba fue el HW-416-B.

```
Serial.begin(115200);
pinMode(PIR_PIN, INPUT);  // PIR_PIN = 27

bool motion = digitalRead(PIR_PIN) == HIGH;
Serial.println(motion ? "MOVIMIENTO" : "SIN MOVIMIENTO");
```

La entrada digital se interpreta como 1 cuando la salida está en HIGH y como 0 cuando está en LOW. Serial a 115200 baud permite verificar la lectura desde la computadora. El PIR informa cambios asociados al movimiento; no confirma por sí solo la presencia de una persona inmóvil.

6.1 Firmware PIR: comunicación y validación (continuación)

Comunicación y toma de muestras

La versión MQTT utiliza WiFi.h para conectar el ESP32 a la red y PubSubClient para publicar en Mosquitto. El archivo de configuración define las credenciales, la IP del servidor, el puerto 1883 y el tópico presence/pir/telemetry. El inicio de las conexiones se realiza en setup:

```
WiFi.mode(WIFI_STA);
WiFi.setAutoReconnect(true);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
mqttClient.setServer(MQTT_HOST, MQTT_PORT);
```

El programa mantiene las conexiones y toma una lectura cada 200 ms nominales mediante millis(). Imprime cada muestra por Serial, aunque el estado no cambie, y la publica cuando hay conexión:

```
if (now - lastSample >= SAMPLE_INTERVAL_MS) {
  lastSample = now;
  bool motion = digitalRead(PIR_PIN) == HIGH;
  Serial.println(motion ? "MOVIMIENTO" : "SIN MOVIMIENTO");
  if (WiFi.status() == WL_CONNECTED && mqttClient.connected()) {
    publishReading(motion);
  }
}
```

Cada mensaje JSON identifica el caso y la placa, e incluye el estado lógico y su valor numérico. Ejemplo de una lectura con movimiento:

```
{"case_id":"pir","source":"pir-<id>",
 "presence":true,"raw_value":1}
```

```
mqttClient.publish(MQTT_TOPIC, payload, false);
```

El backend valida el mensaje, registra la hora de recepción, almacena la lectura y actualiza la web. El firmware reintenta las conexiones cada 5 segundos. Publica con QoS 0 y sin retain; no conserva las muestras para reenviarlas tras una desconexión. Los reintentos pueden demorar el muestreo, por lo que 200 ms no es una garantía de tiempo real estricto.

Problemas encontrados y alcance del avance

En macOS, el error de compatibilidad de la herramienta ctags se resolvió con Rosetta. La carga falló inicialmente a 921600 baud y se completó al reducir la velocidad a 115200. Los caracteres ilegibles del monitor se corrigieron configurándolo también a 115200 baud. Para cargar el programa, el puerto debe estar disponible y corresponder a la placa conectada.

Tras revisar el cableado y repetir las pruebas, se confirmaron lecturas 0 y 1 tanto por Serial como en la web mediante MQTT. Arduino IDE se utiliza para cargar o modificar el firmware; después, el ESP32 ejecuta el programa mientras tenga alimentación. Esta comprobación verifica el flujo de datos, pero todavía no determina precisión, falsos positivos, falsos negativos ni latencia medida.

6.2 Firmware desarrollado para prueba de obtención de señal CSI

Como primer paso del Caso 2 se desarrolló un programa para el ESP32 (PlatformIO, framework Arduino) que conecta la placa como cliente a la red Wi-Fi del router (2,4 GHz) y activa la captura de CSI mediante las funciones del SDK de Espressif (esp_wifi_set_csi_config, esp_wifi_set_csi_rx_cb y esp_wifi_set_csi).

El CSI se obtiene de los paquetes que el ESP32 recibe. Para asegurar un flujo constante, el programa envía un ping al router cada 100 ms, y cada respuesta genera un paquete CSI. Se conservan solo los paquetes cuyo origen es la MAC del router. Cada paquete se envía por el puerto serie a la computadora en una línea de texto, con los metadatos de recepción y los valores del CSI.

Dificultades: Durante las primeras pruebas, la conexión falló por baja señal y se resolvió acercando la placa al router.



Figura 5: Conexión exitosa con el router WIFI

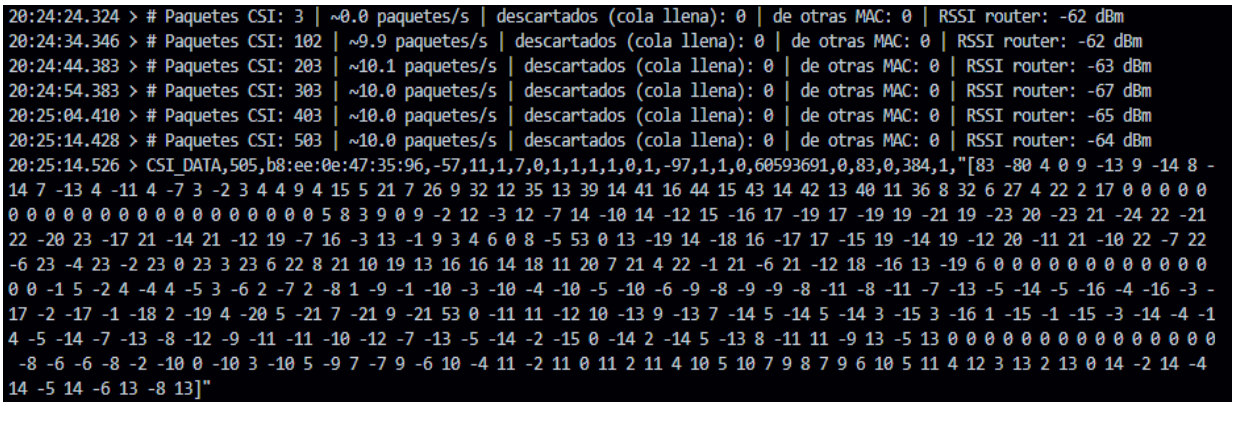


Figura 6: Salida del ESP32 por el monitor serie.

En la Figura 6 se observa que se logró recibir unos 10 paquetes por segundo, en coincidencia con la frecuencia del ping de 100ms (se observa en Figura 5).

Cada paquete CSI tiene dos partes:

- **Metadatos de recepción:** datos que el ESP32 agrega sobre cómo recibió el paquete. Entre ellos están la MAC del router (que permite descartar paquetes de otros dispositivos), el RSSI y una marca de tiempo en microsegundos. No se usan para detectar presencia, pero sirven para filtrar y ordenar las muestras.
- **CSI propiamente dicho:** es la parte que se procesará para detectar presencia. Contiene un valor complejo por cada subportadora, guardado como dos bytes (parte imaginaria y parte real). Cada valor indica cuánto se atenuó y desfasó la señal en esa frecuencia al recorrer el ambiente, por lo que cambia cuando una persona se mueve entre el router y el ESP32.

En los paquetes observados, los 384 bytes de CSI equivalen a 192 pares imaginario-real distribuidos entre los campos LTF; no representan 192 subportadoras útiles distintas. El procesamiento deberá seleccionar los índices válidos según el paquete. Los primeros cuatro bytes se descartarán cuando first_word_invalid sea verdadero; los valores cero no se eliminarán indiscriminadamente [9].

* RSSI (*Received Signal Strength Indicator*, o Indicador de Intensidad de Señal Recibida): medida de la potencia de la señal de radiofrecuencia que recibe un dispositivo.

Próximos pasos

• Registrar series con el ambiente vacío y con personas quietas y en movimiento.
• Integrar la captura CSI real con el procesamiento de amplitud, filtrado y varianza; validar la selección de muestras y ajustar los umbrales con ensayos.
• Publicar las muestras por MQTT hacia Mosquitto e integrarlas en el backend, que enviará los resultados a la web para compararlos con el PIR.

6.3 Interfaz Web

Se desarrolló una interfaz web propia para visualizar las lecturas, consultar el historial y comparar los tres métodos del proyecto. Se eligió esta alternativa por su flexibilidad para representar los ensayos PIR y CSI. En este avance se verificó el Caso 1 con el sensor real; la comparación experimental con los casos CSI sigue pendiente.

Arquitectura del sistema

PIR → ESP32 → Wi-Fi/MQTT → Mosquitto → Python/FastAPI
→ validación y almacenamiento SQLite → REST/WebSocket → React

MQTT comunica el dispositivo con el servidor a través de Mosquitto. El backend recibe y valida la telemetría, almacena las lecturas y entrega los datos a la interfaz. WebSocket permite actualizar el estado en vivo; REST se utiliza para consultar registros y gestionar los ensayos. El navegador se comunica con FastAPI, sin suscribirse directamente al broker MQTT.

6.3 Interfaz Web: tecnologías elegidas (continuación)

Frontend: React, TypeScript y CSS

React permite organizar la interfaz en componentes reutilizables y actualizar las vistas cuando llegan nuevas lecturas. TypeScript agrega comprobaciones de tipos sobre JavaScript y ayuda a mantener consistentes los datos recibidos del servidor. Vite facilita el desarrollo y la compilación; React Router organiza la navegación. CSS define la presentación y adapta las pantallas a computadoras y dispositivos móviles. Apache ECharts representa señales temporales y comparaciones entre métodos.

Backend: Python y FastAPI

Se eligió Python por la disponibilidad de herramientas para procesamiento numérico, como NumPy y SciPy, necesarias para trabajar con CSI. FastAPI reúne la API REST y la comunicación WebSocket. Pydantic valida la estructura de los mensajes recibidos y el cliente paho-mqtt conecta el servidor con Mosquitto. Esta separación permite modificar el procesamiento sin incorporar esa lógica a las pantallas.

Comunicación y persistencia

MQTT desacopla los nodos que publican telemetría del backend que la recibe. Mosquitto funciona como intermediario y permite utilizar el mismo canal de adquisición para los tres casos. SQLite guarda lecturas y ensayos en una base local sencilla, adecuada para esta primera etapa y sin requerir otro servidor. El backend concentra el acceso a esos registros y los expone a la web mediante REST y WebSocket.

Prueba USB y alcance de la validación

Además del flujo MQTT, se conserva una página de diagnóstico que lee Serial por USB mediante Web Serial en navegadores compatibles. En esa prueba, la lectura en vivo se realiza desde el navegador y el guardado se solicita al backend. Esta vía permite revisar el sensor y el cableado; el flujo integrado del proyecto utiliza Wi-Fi, MQTT y FastAPI.

Con el PIR real se confirmaron estados 0 y 1 por Serial y en la web mediante MQTT. El sistema permite registrar ensayos con una referencia de presencia indicada por el usuario para calcular métricas; las lecturas del sensor por sí solas no demuestran exactitud. No se presentan resultados simulados ni se afirma haber validado aún la detección CSI o el desempeño comparativo de los tres casos.

Repositorio del código: github.com/tpII/2026-m1-deteccion-presencia.

7. Documentación Relacionada

Se conserva la siguiente captura de la interfaz durante su desarrollo inicial. Es documentación visual histórica y no representa un resultado experimental ni la versión actual de la prueba MQTT con hardware.



La bitácora registra el desarrollo, las dificultades y las decisiones del proyecto:

https://github.com/tpII/2026-m1-deteccion-presencia/blob/main/BITACORA.md

Pendiente de adjuntar: fotografías del conexionado del PIR y capturas actualizadas de las lecturas reales en la web. La documentación en video se encuentra en la sección 8.

8. Documentación en Video

https://mega.nz/folder/a4QX1JST#5PCNEZzSS_gXMbgMWVT8-Q

9. Bibliografía

[1] HUAWEI. What Is CSI Sensing? Huawei Technical Support.
Disponible en:
https://info.support.huawei.com/info-finder/encyclopedia/en/CSI+Sensing.htmll

[2] ESPRESSIF. ESP-CSI: Applications based on Wi-Fi CSI (Channel State Information), such as indoor positioning, human detection. GitHub, 2021.
Disponible en: https://github.com/espressif/esp-csi

[3] ESP32 en Mercado Libre

[4] sensor PIR en Mercado Libre

[5] router WIFI en Mercado Libre

[6] cable USB en Mercado Libre

[7] cables dupont en Mercado Libre

[8] fuente 5V 2A en Mercado Libre

[9] ESPRESSIF. ESP-IDF Programming Guide: Wi-Fi Vendor Features, sección Wi-Fi CSI. Documentación técnica oficial.
Disponible en:
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/wifi-driver/wifi-vendor-features.html